HCMIU Map Collaborative

A Multi-Model Database-Driven Campus Map with Real-Time Collaboration

Huynh Tran Khanh
Student ID: MITIU25210

Advanced Database System (IT502IU)
International University — VNU-HCM

March 12, 2026

Good morning/afternoon. My name is Huynh Tran Khanh. Today I will present my individual DBMS application project: HCMIU Map Collaborative—a web-based campus map platform that combines indoor navigation with a collaborative knowledge graph, all powered by ArangoDB's multi-model architecture. The presentation will cover the motivation, system design, key features, database design, a live demonstration, and conclusions.

Outline

Here is the roadmap for the next eight minutes. We start with motivation and problem context, then move into system architecture and the database model. After that I will walk through the four main feature groups: campus navigation, collaborative knowledge graph, the trial system, and the research toolkit. We will look at screenshots of the running system, discuss the technology stack, and wrap up with conclusions and future work.

Problem Context

Challenge: Campus information systems must handle

- **High schema variability** — rooms, notes, comments, trials, moderation records evolve over time
- **High relationship density** — references, follows, replies, dispute participants form a dense graph

Traditional approach: Polyglot persistence (separate document store + graph DB) $\Rightarrow$ operational overhead, synchronisation burden [1, 2]

Our Approach

Use **ArangoDB**—a single multi-model engine that natively supports documents, graphs, and key-value workloads—to reduce impedance mismatch while keeping full graph query power.

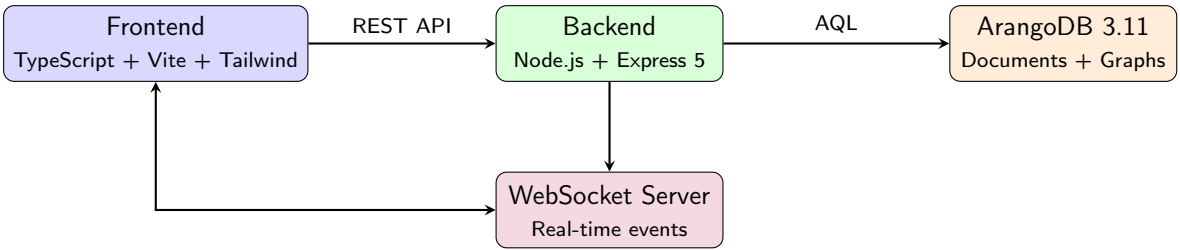
The core problem is that campus information is both highly variable in structure—new entity types keep appearing—and highly connected. Traditional solutions either use a single relational database, which struggles with graph queries, or polyglot persistence, which introduces operational complexity. Our approach is to use ArangoDB, a multi-model DBMS that handles documents and graphs in one engine, eliminating cross-system coordination. This is consistent with Stonebraker's argument that one-size-fits-all is rarely optimal, but here a multi-model engine can cover both needs.

Project Objectives

1. **Interactive campus navigation** — 7-floor indoor map with BFS shortest-path and Traveling Salesman (Held–Karp) multi-stop optimiser
2. **Collaborative knowledge graph** — entity-based discussions with cross-references, follows, notifications, and an activity feed
3. **Transparent dispute resolution** — a structured "Court of Justice" trial system with judge negotiation and voting
4. **Graph-native research tools** — reference discovery, full-text search, degree-of-separation queries
5. **Real-time experience** — WebSocket event propagation for live updates

These are the five concrete objectives. First, we build an interactive 7-floor campus map with two pathfinding algorithms: BFS for point-to-point shortest path, and the Held-Karp bitmask DP algorithm for the Traveling Salesman Problem. Second, we create a collaborative knowledge graph where every room, user, post, and comment is an entity connected by typed edges. Third, we implement a court trial system for dispute resolution. Fourth, we provide research tools that exploit the graph model. And fifth, everything updates in real time via WebSockets.

System Architecture



- **Deployment:** Docker Compose (3 services: DB, backend, frontend)
- **Frontend:** Hyperscript DOM library + Tailwind CSS + Anime.js
- **Auth:** Client-side Argon2i (libsodium) $\rightarrow$ server SHA-256

The architecture follows a clean three-tier pattern. The frontend is a TypeScript single-page application built with Vite and styled with Tailwind CSS. It communicates with the Express 5 backend via REST and also maintains a persistent WebSocket connection for real-time updates. The backend talks to ArangoDB using AQL—ArangoDB's query language. Everything deploys as a Docker Compose stack with three services. Authentication uses a secure SRP-inspired protocol: passwords are hashed client-side with Argon2i via libsodium, then the server applies an additional SHA-256 layer.

Multi-Model Database Design

Document Collections:

- `users` — credentials, salts, tokens
- `entities` — posts, comments, map locations, trial entities
- `follows` — user $\rightarrow$ entity subscriptions
- `notifications` — per-user alerts
- `trials` — court cases with status, judges, votes

Edge Collection:

- `entity_references` — directed edges connecting entities that reference each other

Why Multi-Model?

- Document storage for flexible schema evolution
- Graph edges for relationship traversal
- Single AQL query language for both
- No cross-system synchronisation needed

The database design is the heart of this project. We have five document collections for structured data storage and one edge collection for the knowledge graph. The entities collection is polymorphic—it stores posts, comments, map locations, and trial entities with a type discriminator. The edge collection entity_references stores directed links between entities, enabling graph traversal queries. The key advantage is that both document queries and graph traversals use the same AQL language, eliminating the impedance mismatch that would occur with separate database systems.

Representative AQL Queries

1. Reference Discovery (Graph Traversal):

```
FOR v, e IN 1..1 OUTBOUND @entityId entity_references
  RETURN { entity: v, edge: e }
```

2. Degree of Separation (Shortest Path):

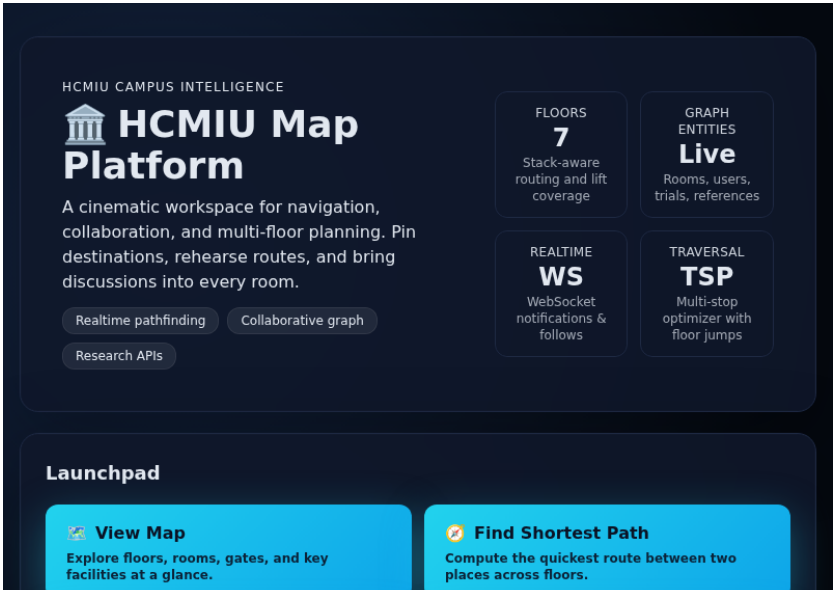
```
FOR v IN OUTBOUND SHORTEST_PATH
  @fromId TO @toId entity_references
  RETURN v
```

3. Full-Text Search:

```
FOR e IN entities
  FILTER CONTAINS(LOWER(e.title), LOWER(@query))
      OR CONTAINS(LOWER(e.body), LOWER(@query))
  RETURN e
```

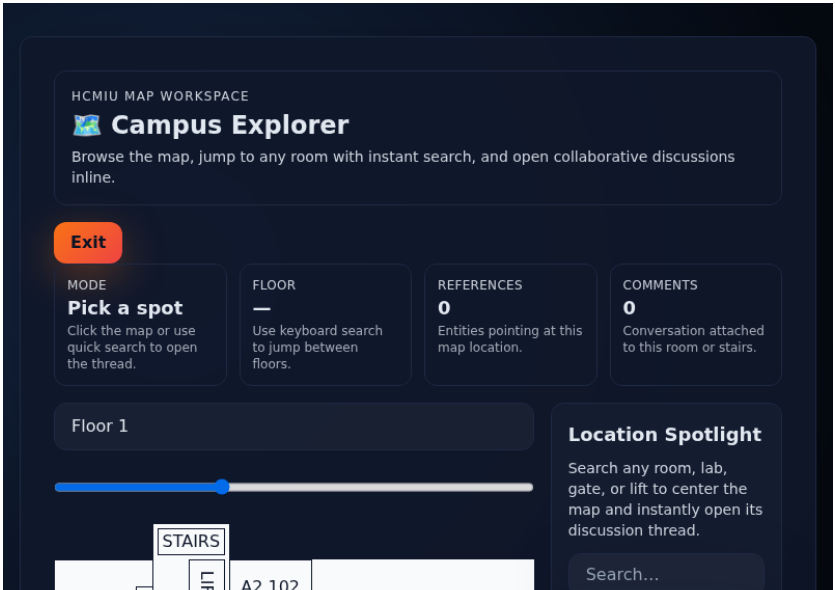
Here are three representative AQL queries that power the research features. The first uses a graph traversal to find all entities referenced by a given entity. The second uses ArangoDB's built-in SHORTEST_PATH function to find the minimum number of hops between any two entities—this is the degree-of-separation feature. The third performs full-text search across entity titles and bodies. Notice how all three queries use the same AQL syntax, whether they are document queries or graph traversals. This uniformity is a major advantage of the multi-model approach.

Landing Page



This is the landing page of the application. It provides four main entry points corresponding to the major features: View Map for floor exploration, Find Shortest Path for point-to-point navigation, Solve Traveling Salesman for multi-stop optimization, and HCMIU Collaborative for the knowledge graph platform. At the bottom, live system signals display the health of the backend API, the collaboration graph, and the wayfinding engine. The design uses a dark theme with color-coded feature cards.

Interactive Campus Map



The map view shows a detailed floor plan of the HCMIU campus. On the left is the interactive SVG map with zoom and pan controls. Users can switch between all seven floors using a dropdown. On the right panel, there is a quick search box with autocomplete for all named rooms. When a room is selected, the panel shows its metadata including reference count and comment count, plus a preview of the discussion thread attached to that location. The "Open in Collaborative" link takes users directly to the full discussion page for that room entity. This bridges the gap between physical navigation and knowledge sharing.

Pathfinding Algorithms

Shortest Path (BFS)

- Breadth-first search on grid graph
- Handles **inter-floor routing** via stairs and lifts
- Visual path overlay on floor maps
- Time complexity: $O(V + E)$

User Flow:
Select start $\rightarrow$ Select end $\rightarrow$
View floor-by-floor directions

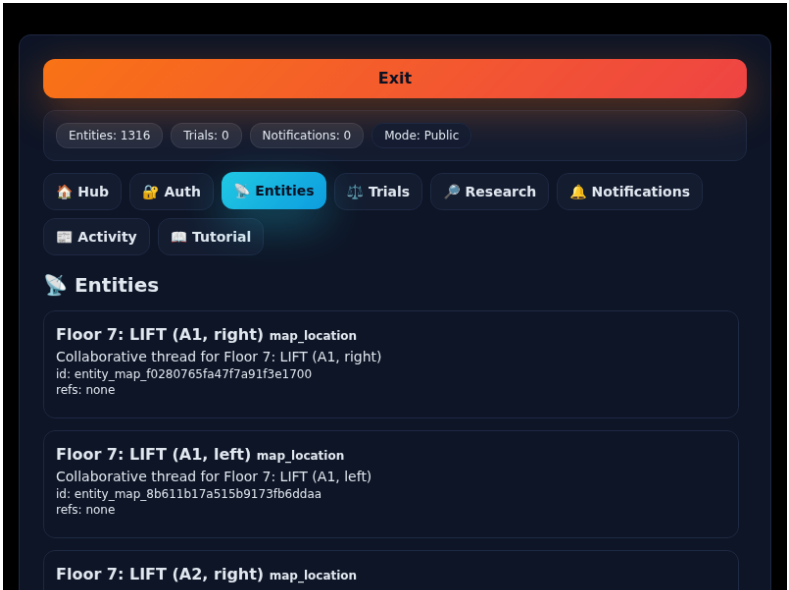
Traveling Salesman (Held–Karp)

- Bitmask dynamic programming
- Optimal visit order for n destinations
- Multi-floor route planning
- Time complexity: $O(2^n \cdot n^2)$

User Flow:
Add locations $\rightarrow$ Solve $\rightarrow$
View optimised route with total distance

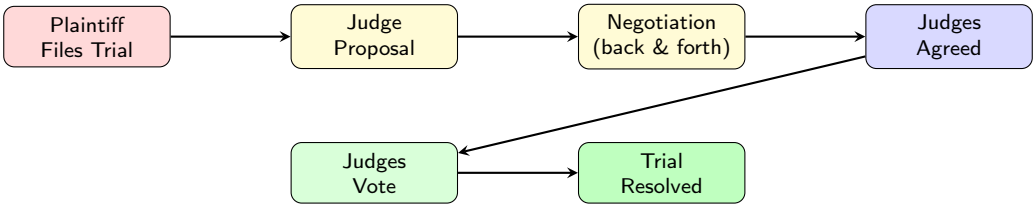
The system implements two pathfinding algorithms. The shortest path finder uses BFS on the campus grid graph. It automatically handles inter-floor routing by detecting when a path needs to go through stairs or lifts, then showing directions floor by floor. The Traveling Salesman solver uses the Held-Karp algorithm—a bitmask dynamic programming approach with time complexity O of 2-to-the-n times n-squared. This finds the optimal visit order for multiple destinations. Both algorithms visualise results as colored path overlays on the floor maps, making it easy to follow directions physically.

Collaborative Knowledge Graph



The collaborative page is the social backbone of the platform. Every element in the system—rooms, users, posts, comments, trials—is treated as a first-class entity in the knowledge graph. Users can create discussion posts, attach comments to any entity, and add cross-references to build a connected knowledge base. The follow system lets users subscribe to entities and receive notifications when new comments are added. All changes propagate in real time via WebSockets—when one user posts a comment, all connected clients see it instantly. The navigation tabs at the top provide access to the Hub, Auth, Entities, Trials, Research, Notifications, and Activity sections.

Court of Justice — Trial System



- Plaintiff files trial against defendant over a disputed entity
- Both parties negotiate judge selection (proposal $\leftrightarrow$ counter-proposal)
- Agreed judges cast votes: `plaintiff` / `defendant` / `no_winner`
- Comments tagged by role: `[plaintiff]`, `[defendant]`, `[judge]`, `[spectator]`

The Court of Justice is one of the most innovative features. It provides a structured workflow for resolving disputes about campus knowledge. A plaintiff files a trial against a defendant, specifying the entity under dispute. Then the two parties negotiate on who the judges should be through a back-and-forth proposal mechanism. Once both sides agree, the judges cast their votes. Throughout the process, all participants can comment, and their comments are tagged with their role. This creates a transparent, auditable record of the dispute resolution. The trial entity itself becomes part of the knowledge graph, maintaining full traceability.

Research & Knowledge Graph Tools

Reference Discovery

- Query entities by ID
- View all outbound references
- Explore the citation network

Full-Text Search

- Search across all entity titles and bodies
- Keyword-based discovery
- Cross-entity relationship finding

Degree of Separation

- Find shortest reference path between any two entities
- Uses ArangoDB's native `SHORTEST_PATH` function
- Visualises intermediate hops

Mention Discovery

- Find which entities mention a given entity
- Reverse reference traversal
- Map cross-entity relationships

The Research section provides four powerful tools that exploit the graph model. Reference Discovery lets users look up any entity and see all the entities it references—essentially browsing the citation network. Full-Text Search performs keyword searches across all entity titles and bodies, useful for discovering related discussions. Degree of Separation uses ArangoDB's native SHORTEST_PATH function to find the minimum number of reference hops between any two entities. This is analogous to the "six degrees of separation" concept but applied to campus knowledge. Mention Discovery performs reverse traversal—finding which entities reference a given entity, revealing unexpected connections.

Real-Time Updates via WebSocket

Event Types:

- `entity.created` — new post or comment
- `entity.updated` — edit to existing entity
- `entity.deleted` — entity removed
- `trial.created` — new trial filed
- `trial.updated` — vote or status change
- `notification.created` — user alert

Design Decisions

- RFC 6455 WebSocket protocol
- Token-authenticated connections
- Server-push model (no polling)
- All connected clients auto-refresh

Result: When User A posts a comment, User B sees it instantly without page refresh—verified in multi-user E2E tests.

Real-time updates are essential for a collaborative platform. We use the WebSocket protocol—RFC 6455—with token-based authentication. When a client connects, it provides its bearer token as a query parameter. The server then pushes events to all relevant connected clients whenever data changes occur. There are six event types covering entity CRUD operations, trial updates, and notifications. This was verified through automated multi-user end-to-end tests using Puppeteer, where two browser instances interact simultaneously and changes appear instantly on both screens.

Technology Stack & Testing

Technology Stack:

Layer	Technology
Frontend	TypeScript, Vite, Tailwind
UI	Hyperscript, Anime.js
Backend	Node.js, Express 5
Database	ArangoDB 3.11
Real-time	WebSocket (ws)
Auth	libsodium (Argon2i)
Deploy	Docker Compose

Testing & Validation:

- **API unit tests** — all REST endpoints
- **E2E tests** — full Docker Compose stack
- **8 demo videos** — Puppeteer-automated screen recordings covering:
  - Map navigation
  - Multi-user real-time
  - Trial system workflow
  - Research tools (4 demos)
  - General platform overview

The technology stack was chosen for developer productivity and architectural simplicity. TypeScript provides type safety on the frontend. Express 5 is the latest major version with improved async support. ArangoDB 3.11 gives us native multi-model capabilities. WebSocket via the ws library provides efficient real-time communication. For testing, we have API unit tests for all REST endpoints, full end-to-end tests using Docker Compose, and eight automated demo videos generated with Puppeteer that exercise every major workflow. These demos serve as both regression tests and documentation—they show exactly how each feature works through real browser interactions.

Conclusions & Future Work

Key Contributions:

1. Demonstrated that a **multi-model DBMS** (ArangoDB) can effectively serve both document storage and graph traversal needs in a single engine
2. Built a **complete campus platform** combining wayfinding, collaboration, governance, and research
3. Implemented **graph-native features** (reference discovery, degree-of-separation, mention traversal) that would be complex in a document-only or relational system
4. Validated correctness through **comprehensive automated testing**

Limitations & Future Work:

- Access control is creator-based; role-based permissions needed
- Event ordering relies on eventual consistency; formal guarantees desirable
- Map data is hardcoded; a map editor would improve extensibility
- Performance benchmarking under concurrent load not yet conducted

To summarise, this project makes four key contributions. First, it demonstrates that ArangoDB's multi-model architecture can handle both document storage and graph queries effectively, reducing the need for polyglot persistence. Second, it delivers a complete campus platform that goes beyond simple maps to include collaboration, governance, and research. Third, graph-native features like degree-of-separation queries showcase capabilities that would be complex in traditional databases. Fourth, comprehensive testing—including 8 automated demo videos—ensures correctness and serves as living documentation. For future work, we identified four areas: role-based access control, formal consistency guarantees, a dynamic map editor, and performance benchmarking under load.

References

- M. Stonebraker and U. Çetintemel, "One Size Fits All: An Idea Whose Time Has Come and Gone," *Proc. ICDE*, pp. 2–11, 2005.
- P. Sadalage and M. Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*, Addison-Wesley, 2012.
- ArangoDB Inc., "ArangoDB—the Multi-Model NoSQL Database," `https://www.arangodb.com/`, 2024.
- A. Zimmerman and M. Martin, "Post-occupancy evaluation: benefits and barriers," *Building Research & Information*, vol. 29, no. 2, pp. 168–174, 2001.

These are the main references that informed the design decisions. Stonebraker's seminal work on the limitations of one-size-fits-all databases motivated our choice of a multi-model approach. Sadalage and Fowler's book on NoSQL provided context on polyglot persistence trade-offs. The ArangoDB documentation was the primary technical reference. And Zimmerman's work on wayfinding informed the navigation design. Thank you for your attention. I am happy to take questions.

Thank You

Questions?

Live Demo: `https://hcmiumap.huynhtrankhanh.com/`

Source Code: `https://github.com/huynhtrankhanh/hcmiu-map-collaborative`

Thank you for listening. The live demo is available at the URL shown. The full source code, including all automated tests and demo video generation scripts, is open-source on GitHub. I am happy to answer any questions about the architecture, database design, algorithms, or implementation details.